
ALCOR
academy
TRAINING PROGRAMME
Lesson 5
Coupling & Cohesion
“It is very hard to provide the degrees of freedom that make computing important, without also providing the means for poor designers to come up with poor designs.”— Alan Kay

Cohesion and Coupling

Cohesion

says **how strongly related and coherent** are the **responsibilities *within*** modules (classes) of an application

hence

it should be **MAXIMIZED**

Coupling

is the **degree of interdependence *between*** modules (classes) of an application

hence

it should be **MINIMIZED**

Cohesion categories

Class Cohesion

Holistic cohesion at *class* level deducted from fields, methods and their interactions.

Method Cohesion

Cohesion of the *individual methods*, focusing on how ‘single-minded’ is a method.

Yes - even *methods should do one and only one thing*.

Class Cohesion

Good



Bad

Ideal ➥ The class doesn't show any mixed cohesion e.g. *Domain Object*

Mixed-Role ➥ The class has one or more fields that relate objects of the class to other objects on the same abstraction layer, but the fields have nothing to do with the underlying semantics of the class e.g. *Domain or Application Service*

Mixed-Domain ➥ The class has one or more fields that relate objects of the class to other objects on a different abstraction layer. Hence the fields are completely unrelated with the underlying semantics of the class e.g. *Orchestrator or Controller in MVC pattern*

Mixed-Instance ➥ The class represents two different types of objects and should be split in two separate modules. Typically different instances uses only subsets of fields and methods of the class e.g. *God Class*

Method Cohesion

Good



Bad

Functional ➥ it performs a single problem-related task

Sequential ➥ it combines two functions in which the output from the first one is used as the input to the second one

Communicational ➥ It combines two functions which access the same data fields to execute

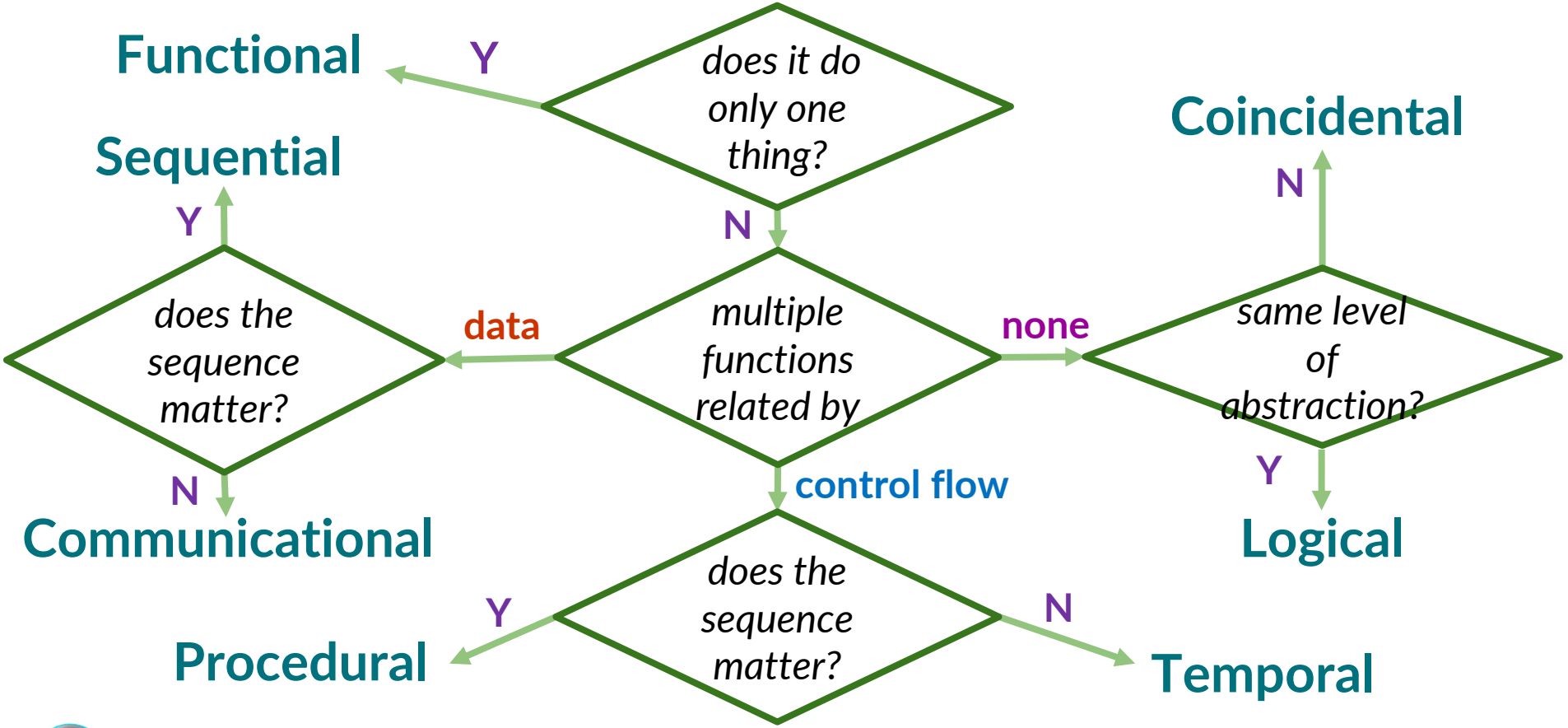
Procedural ➥ It supports multiple weakly related functions

Temporal ➥ It performs multiple functions related only by the moment in time they execute

Logical ➥ It supports multiple functions but the choice of the specific execution is determined by a control parameter passed in input

Coincidental ➥ There is no clearly defined purpose for the method or it performs multiple unrelated functions

Method Cohesion



Coupling categories

Interaction Coupling

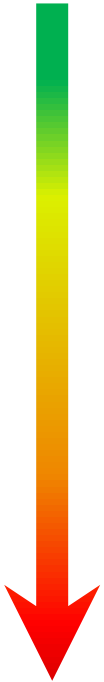
Coupling deriving by class collaborations via message passing (method invocation).

Inheritance Coupling

Coupling deriving by class inheritance hierarchy.

Interaction Coupling

Good



Bad

No coupling ➥ when two classes do not call any method one another.

Data ➥ the calling object passes a variable to the called method. If the variable is not a primitive, all data of the object is used by the called method to perform its function.

Stamp ➥ the calling object passes a composite variable but only a portion of the object's data is used to perform the method's function.

Control ➥ the method receives a variable whose internal value is used to control the execution.

Common or global ➥ the calling method refers to global data sitting outside the individual objects.

Content or pathological ➥ the method of one object uses the properties or methods of another object to perform its function

Inheritance Coupling

- ➔ It's the ***strongest type*** of coupling.
- ➔ If a class inherits from another one it binds itself completely to the superclass. When the superclass definition is modified all the subclasses are affected.
- ➔ That's why a high degree of inheritance coupling ***may not be a good idea.***

so

1) Use inheritance only for ***generalization/specialization*** semantics (***Same-Kind-Of*** relationship)
2) Use ***composition*** over inheritance in any other case

Formula of *class cohesion*

n_F ⇒ number of fields

n_M ⇒ number of methods

n_{MF} ⇒ number of methods accessing the field F

$$\textbf{Cohesion} = \frac{\sum_0^F n_{MF}}{n_M n_F}$$

[0 < *Cohesion* <1]

Cohesion* ⩽ 0.5** ⇒ the class has ***more than one responsibility

Formula of *class coupling*

d_I ⇒ data in input

d_O ⇒ data in output

cd_I⇒ control data input

cd_O⇒ control data output

g ⇒ global data

cg ⇒ control global data

m_I ⇒ fan in (modules call in)

m_O⇒ fan out (modules call out)

k ⇒ control data crappiness constant

$$\textbf{\textit{Coupling}} = 1 - \frac{1}{d_I + d_O + g + k(cd_I + cd_O + cg) + m_I + m_O}$$

[0 < *Coupling* < 1] should stay ⩽ **0.75**

System's cohesion and coupling

is the **AVERAGE**

of ***class cohesion*** and ***class coupling***

n_C ⇒ number of classes

Total Cohesion

$$\frac{\sum_0^C cohesion}{n_C}$$

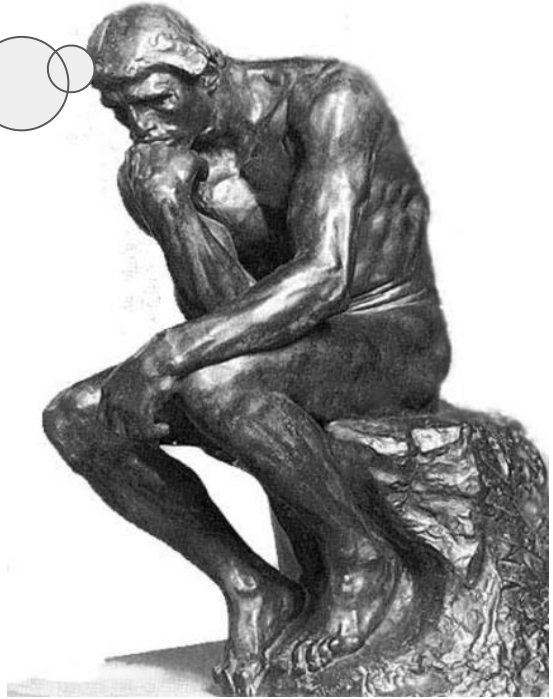
Total Coupling

$$\frac{\sum_0^C coupling}{n_C}$$

Cohesion and Coupling

Yes, but...

SHOW ME SOME CODE!!!



```
public class A{
}
```

Cohesion: 0
Coupling: -Inf ⇒ ***useless***

```
public class A{
    B b = new B();
}
```

Cohesion: 0

Coupling: 0 (1 - 1/1 ⇒ 1 - 1 ⇒ 0)

⇒ ***useless***

```
public class A{
    B b;

    public A(B injected-b){
        b = injected-b;
    }
}
```

Cohesion: 1 (1/1)
Coupling: 0 (1 - 1/1)

```
public class A{
        int x = 1;

        public Method1(int y){
                x = y;
        }
}
```

Cohesion: 1 (1/1)
Coupling: 0 (1 - 1/1)

```
public class A{
        B b = new B();

        public Method1(){
                b.Method1();
        }
}
```

Cohesion: 1 (1/1)
Coupling: 0.5 (1 - ½) ⇒ ***man in the middle***

```
public class A{
        int x = 1;
        private b = new B();

        public Method1(){
                b.Method1(x);
        }
}
```

Cohesion: 1 ((1+1)/1*2 ⇒ 2/2 ⇒ 1)

Coupling: 0.67 (1 - 1/(1+2) ⇒ 1 - ⅓ ⇒ 0.67)

```
public class A{
        int x = 1;
        int y = 2;
        private b = new B();

        public Method1(){
                b.Method1(x, y);
        }
}
```

Cohesion: 1 ((1+1+1)/1*3 ⇒ 3/3 ⇒ 1)

Coupling: 0.75 (1 - 1/(1+3) ⇒ 1 – 1/4 ⇒ 3/4)

```
public class A{
        int x = 1;
        int y = 2;
        int c;
        private b = new B();

        public Method1(){
                b.Method1(x, y);
        }

        public Method2(){
                c = b.Method2();
        }
}
```

Cohesion: 0.625 ((1+1+1+2)/2*4 ⇒ ⅝)

Coupling: 0.75 (1 - 1/(2+1+1) ⇒ 1 - ¼ ⇒ ¾)

```
public class A{
      int x = 1;
      int y = 2;
      private b = new B();
      private c = new C();

      public Method1(){
            b.Method1(x);
      }

      public Method2(){
            c.Method1(y);
      }
}
```

Cohesion: 0.5 [(1+1+1+1)/2*4]

Coupling: 0.75 (1 - 1/(2+2) ⇒ 1-¼ ⇒ ¾)

```
public class A1{
        int x = 1;
        private b = new B();

        public Method1(){
                b.Method1(x);
        }
}
```

```
public class A2{
        int y = 2;
        private c = new C();

        public Method2(){
                c.Method1(y);
        }
}
```

A1 = (1 + 1)/2*1 = 2/2 = **1**; **A2** = (1 + 1)/2*1 = **1**

Average Cohesion: **1**

(**A1**=1-1/(1+1) = 1 - ½ = **0.5**; **A2**=1-1/2 = **0.5**)

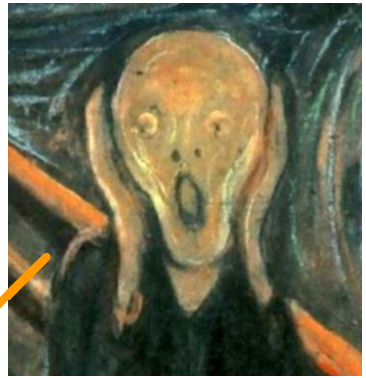
Average Coupling: **0.5**

```
public class A{
        int x = 1;
        int y = 2;
        private b = new B();

        public Method1(){
                b.Method1(x);
        }

        public Method2(){
                C c = b.Method2();
                //b.Method2().Method1(y)
                c.Method1(y);
        }
}
```

Demeter



Cohesion: 0.67 ((1+1+2)/2*3)

Coupling: 0.8 (1 - 1/(2+1+2))

```
public class A{
        B b = new ClassB();
        int x = 1;

        public Method1(bool flowControl){
                if(flowControl)
                        b.Method1(x);
                else
                        b.Method2(x);
        }
}
```

Cohesion: 1 ((1+1)/1*2)

Coupling: 0.83 (1 - 1/((4*1)+1+1), with k=4)

Single Responsibility Principle

Maximize ***Cohesion***

Class collaboration

Minimize ***coupling***

Rationalize interactions

Method Coupling Premise

Method Coupling Premise

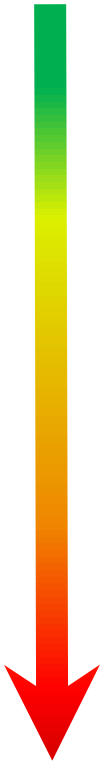
-

Less

COUPLING

More

+



```
0   ➢ void Method()
1   ➢ void Method(int x)
1   ➢ int Method()
1+1 ➢ void Method(ClassA a)
1+1 ➢ ClassA Method()
2   ➢ void Method(int x, int y)
2   ➢ int Method(int x)
2+2 ➢ void Method(ClassA a, ClassB b)
2+2 ➢ ClassA Method(ClassB b)
...
```